

KT 사례기반 AI 교육

- VOC 고객 해지 예측 모델링

1. AIDU 기본 사용법 및 데이터 분석 준비하기(완료)

2. 데이터 탐색하기(완료)

3. 데이터 전처리하기

4. 머신러닝 모델 구현

5. 딥러닝 심층신경망 모델 구현

```
In [ ]: # 코드실행시 경고 메시지 무시

import warnings
warnings.filterwarnings(action='ignore')
```

```
In [ ]: # 필요 라이브러리 임포트 하기
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

```
In [ ]: !pip install seaborn
```

```
In [ ]: import matplotlib.font_manager as fm

font_list = []
font_list.append(fm.findSystemFonts(fontpaths=None, fontext='ttf'))
font_list
```

```
In [ ]: plt.rc('font', family='NanumGothicCoding')
sns.set(font="NanumGothicCoding",
        rc={"axes.unicode_minus":False}, # 마이너스 부호 깨짐 현상 해결
        style='darkgrid')
```

```
In [ ]: # voc 데이터 읽어 들이기
df = pd.read_csv("../voc_rcp_practice2.csv")
df
```

Chapter 3. 데이터 전처리하기

1. 결측치 처리하기
2. 이상치 처리하기
3. 데이터 시각화하기

4. Feature Engineering

1. 결측치 처리하기

- ① 결측치 처리 시 주의사항
- ② 결측치 채우기
- ③ 결측치 제거하기

주의!!

① 결측치 처리 시 주의사항

데이터 결측치 처리시 반드시 원본 데이터를 COPY 해서 사용해야 합니다.

파이썬에서 Copy 메소드를 사용하지 않으면 주소값을 복사해서 사용기 때문에 원본 값을 변경 시키게 됩니다.

따라서 원본 항목을 보전하면서 데이터를 보정하려면 copy 메소드를 사용 해주셔야 합니다.

```
In [ ]: # df Data 복사
df_fix = df.copy()
df_fix.head()
```

```
In [ ]: df_test=df
df_test.head()
```

```
In [ ]: df_test['age']=0
df_test.head()
```

```
In [ ]: # df_test 데이터 변경 후 df 데이터 확인
df.head()
```

```
In [ ]: #Copy 메소드로 변경한 df_fix 데이터 확인
df_fix.head()
```

```
In [ ]: df=df_fix.copy()
```

② 결측치 채우기

• 단일 값으로 채우기

사용자가 지정하는 단일값으로 채워서 결측치를 처리할 수 있습니다.

```
In [ ]: #fillna 함수를 사용해서 특정 숫자나 문자로 결측치를 처리하는 방법
df=df.fillna(15)
df.head()
```

```
In [ ]: df.isnull().sum()
```

- 주변값을 활용하여 결측치 채우기

fillna()의 method 파라미터를 사용하면, 사용자가 지정하는 일괄적인 값이 아닌 주변값으로 채울 수 있습니다.

```
In [ ]: # 뒤에 있는 data를 사용해서 결측치를 처리하는 방법
df=df_fix.copy()
df=df.fillna(method='backfill')
df.head()
```

```
In [ ]: # 앞에 있는 data를 사용해서 결측치를 처리하는 방법
df=df_fix.copy()
df=df.fillna(method='ffill')
df.head()
```

주의!!

단, method 파라미터사용시 첫 Record 또는 마지막 Record가 결측치 인지 확인해야 합니다.

- replace()함수로 결측치 채우기

```
In [ ]: df=df_fix.copy()
df['age_itg_cd']=df['age_itg_cd'].replace(np.nan, df['age_itg_cd'].median())
df.head()
```

③ 결측치 제거하기

결측치를 채우기로 처리하다보면 경우에 따라 값의 왜곡이 크게 발생하는 발생 하는 경우가 있습니다.

이때는 제거하기 기법을 사용하여 결측치를 처리하면 데이터의 적합성을 보존 할 수 있습니다.

- listwise 방식으로 제거하기

record의 항목 중 1개의 값이라도 NA이면 해당 데이터 행 전체 제거

```
In [ ]: #listwise 방식으로 제거 하기
df=df_fix.copy()
df=df.dropna()
df.info()
```

- pairwise 방식으로 제거하기

모든 항목이 NA인 데이터 행만 제거

```
In [ ]: #pairwise 방식으로 제거하기
df=df_fix.copy()
df=df.dropna(how='all')
df.info()
```

- 임계치를 설정해서 제거하기

NA의 갯수에 따라 결측데이터를 판단하고자 한다면, thresh 파라미터를 사용하면 됩니다.

```
In [ ]: #임계치를 설정해서 제거하기
df=df_fix.copy()
df=df.dropna(thresh=14)
df.info()
```

- 특정 열에 있는 NA만 참고하여 결측치를 제거

Subset 파라미터를 사용하면 됩니다.

```
In [ ]: # 특정열 안에 결측치가 있을 경우 삭제하기
df=df_fix.copy()
df=df.dropna(subset=['cust_clas_itg_cd'])
df.info()
```

2. 이상치 처리하기

- ① 범주형 데이터 이상치 처리하기
- ② 수치형 데이터 이상치 처리하기

① 범주형 데이터 이상치 처리하기

범주형 데이터의 경우 value_counts 메소드를 사용하면, 값의 분포와 함께 Trash 값이 있는지도 확인 가능합니다.

```
In [ ]: df['cust_clas_itg_cd'].value_counts()
```

'cust_clas_itg_cd' 값에는 'L, G, I, J, F, O, H, M, V, R'이 들어있는것을 확인할수 있습니다. 실제로 그렇지 않지만 'R'이 실제로는 존재하지 않는 값이라고 가정해 봅시다. 이상치를 처리하는 방법은 제거하기와 변경하기가 있습니다.

- 이상치 제거하기

```
In [ ]: df_data=df[(df['cust_clas_itg_cd']!='R')]
print(df_data['cust_clas_itg_cd'].value_counts())
```

- 이상치 변경하기

```
In [ ]: df_data=df.copy()
df_data['cust_clas_itg_cd']=df_data['cust_clas_itg_cd'].replace('R','L')
print(df_data['cust_clas_itg_cd'].value_counts())
```

② 수치형 데이터 이상치 처리하기

describe 함수를 사용해서 수치형 변수의 기술 통계를 확인 할 수 있습니다.

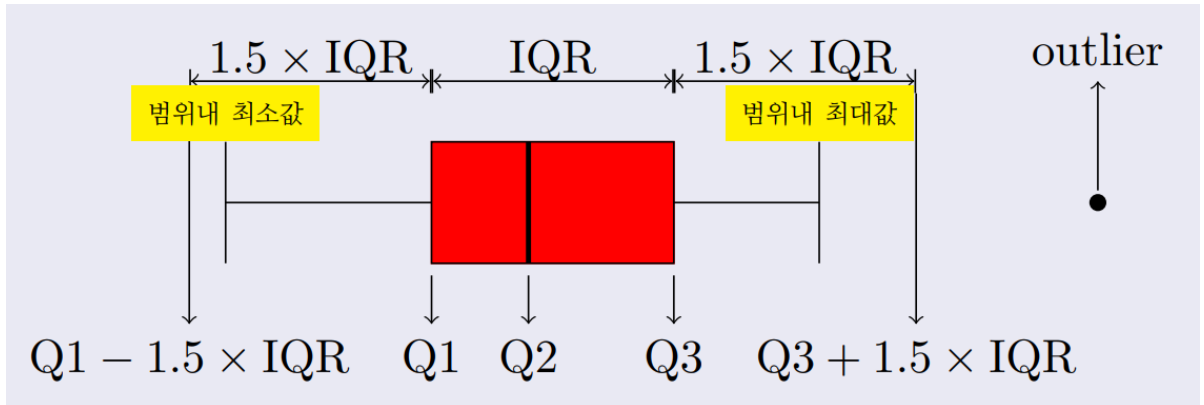
```
In [ ]: df.describe()
```

[참고]

Q1와 Q3가 거리의 1.5배가 넘어가는 값을 Outlier 라고 합니다.

이 값들은 이상치로써 일반적으로 제거 또는 변경하여 데이터를 분석,학습 합니다.

단, 이상치 분석시에는 제거하지 않습니다.



[출처] <https://wikidocs.net/166860>

Median(중앙값) : 중앙값의 50% 위치

Mean(평균값) : 평균값

Box(박스) : Q1(25%) ~ Q3(75%) 까지 값들, IQR(Interquartile Range)

Whiskers(수염) : Q1, Q3 로 부터 IQR 거리의 1.5배 내에서 가장 멀리 떨어진 데이터 점까지 이어져 있는 구역

Outlier(이상치) : Whiskers 보다 바깥 쪽에 데이터가 존재하면, 통계적으로 이상치로 분류

• 차트를 그려서 이상치 확인하기

이상치는 boxplot을 그려서 확인이 가능합니다.

```
In [ ]: plt.figure(figsize=(16,6))
sns.boxplot(df['age_itg_cd'])
plt.show()
```

• 이상치 제거하기

```
In [ ]: #Q1, Q3 구하기
q1 = df['age_itg_cd'].quantile(0.25)
q3 = df['age_itg_cd'].quantile(0.75)

# 1.5 * IQR(Q3 - Q1)
iqr = 1.5 * (q3 - q1)

# 이상치 제거하기
df_data=df[(df['age_itg_cd'] < (q3 + iqr)) & (df['age_itg_cd'] > (q1 - iqr))]

df_data.describe()
```

- 이상치 변경하기

```
In [ ]: # 이상치 대체값 설정하기
age_min = q1 - iqr
age_max = q3 + iqr

# 이상치를 변경
df[(df['age_itg_cd'] > age_max)] = age_max
df[(df['age_itg_cd'] < age_min)] = age_min

df.describe()
```

```
In [ ]: df = df_fix.copy()
```

[실습1] 데이터 전처리하기

[Q] 'df'에서 수치형 컬럼의 결측치를 중간값(median)으로 채워 주세요.

```
In [ ]: # 수치형 컬럼 결측치 중앙값으로 채우기
df['age_itg_cd'].fillna(df['age_itg_cd'].median(), inplace=True)
```

```
In [ ]: df.isnull().sum()
```

[Q] 'df'에서 Object형 컬럼의 결측치는 최빈값으로 채워 주세요.

```
In [ ]: #value_counts() 함수로 최빈값 찾기
df['cust_clas_itg_cd'].value_counts()
#---> value_counts()는 null 값을 제외하고 카운팅 합니다.
```

```
In [ ]: # fillna() 최빈값으로 채우기
df['cust_clas_itg_cd'].fillna('L', inplace=True)
df['cust_clas_itg_cd'].value_counts()
```

```
In [ ]: #value_counts() 함수로 최빈값 찾기
print(df['cont_sttus_itg_cd'].value_counts())
print(df['cust_dtl_ctg_itg_cd'].value_counts())
print(df['voc_mis_pbis_yn'].value_counts())
```

```
In [ ]: df['cont_sttus_itg_cd'].fillna('10001', inplace=True)
df['cust_dtl_ctg_itg_cd'].fillna('10003', inplace=True)
df['voc_mis_pbis_yn'].fillna('N', inplace=True)
```

```
In [ ]: df.isnull().sum()
```

[Q] 'df' 데이터에서 추가적으로 전처리 대상으로 고려할 사항이 있는지 데이터 컬럼을 살펴보면서 확인해 보세요.

- 컬럼 중 대부분 같은 값을 가져서 데이터 분석에 의미가 없는 컬럼
- 고객 ID, 상품 ID, 날짜 등 유니크하거나 다양한 키 값을 가져 데이터 값을 가진 컬럼

- 'voc_mis_pbis_yn' 컬럼의 경우 'N' 값이 99% 이상 차지하므로 삭제

```
In [ ]: df['voc_mis_pbis_yn'].value_counts(normalize=True)
df.drop(columns=['voc_mis_pbis_yn'], inplace=True)
```

- 고객ID, 상품ID 등 유니크하거나 다양한 키 값을 가진 컬럼 삭제

```
In [ ]: print(df['voc_prod_sbt_id'].value_counts())
print(df['voc_wjt_sorc_id'].value_counts())
print(df['bprod_sbt_id'].value_counts())

df.drop(columns=['voc_prod_sbt_id', 'voc_wjt_sorc_id', 'bprod_sbt_id'], inplace=True)
```

```
In [ ]: df_fix = df.copy()
```

3. 데이터 시각화하기

- ① 기본 사용법
- ② 목적별 차트 그리기
- ③ 차트 꾸미기

데이터 시각화는 데이터 분석과정에서 매우 중요한 부분입니다.

프로젝트 초기 단계 또는 리포팅 단계에서의 탐색적 데이터 분석(EDA)의 일환으로 결과를 다른 사람과 공유할 때 유용합니다.

또한 데이터를 시각화 하면 값의 분포나 값 사이의 관계를 확인하고 모델링을 위한 가설 도출을 하는데 도움이 됩니다

① 기본 사용법

데이터 시각화를 하기 위해서는 아래 3가지 코드를 기억해야 합니다.

- plt.figure() - 시각화 그림을 표시할 영역 지정
- plt.plot() - 시각화 차트 및 값 지정
- plt.show() - 시각화 명령어

```
In [ ]: # Matplotlib를 사용하여 간단한 차트를 그리기
plt.figure()
plt.plot([1,2,3], [100,120,110])
plt.show()
```

```
In [ ]: # Seaborn을 사용하여 간단한 차트를 그리기
```

```
plt.figure()
sns.lineplot([1,2,3], [100,120,110])
plt.show()
```

주의!!

seaborn을 사용할 때는 반드시 Matplotlib 패키지도 함께 불러와야 합니다.

기본적인 시각화 기능은 Matplotlib 패키지에 의존하게 되지만, 차트마다 함수와 파라미터가 조금씩 다르니 사용할 때 주의가 필요합니다.

② 목적별 차트 그리기

• 선그래프(line plot)

선 그래프는 수치의 변화를 선으로 이어서 그려 줍니다.

이 때문에 보통 선그래프는 시간에 따른 데이터의 변화 추세를 볼 때 사용합니다.

• 산점도(Scatter)

산점도는 키와 몸무게 같은 두 값 간의 관계를 표현 합니다.

두 값이 양의 상관관계인지 음의 상관관계인지를 파악할 수 있습니다.

```
In [ ]: # 'VOC처리소요시간통합코드', '고객체감기준확인소요시간' 간의 관계를 알아보기 위해 산점도 그리기
# Matplotlib
plt.figure(figsize=(6,6))
plt.scatter(y='cust_snsry_base_conf_need_time', x='voc_trt_need_time_itg_cd', data=df)
plt.show()
```

• 히스토그램

히스토그램은 수치형 데이터의 분포를 나타냅니다. 흔히 빈도, 빈도밀고, 확률 등의 분포를 그릴 때 사용합니다.

구간값(bins)을 정해주지 않으면 10개 구간으로 나누어 그래프를 그려 줍니다.

```
In [ ]: # 'age_itg_cd'에 대한 빈도를 10개 구간으로 구간으로 그리기
plt.figure()
plt.hist(df["age_itg_cd"])
plt.show()
```

• 박스 그래프

수치적 자료를 표현하는 그래프입니다.

수치적 자료로부터 얻어낸 통계량인 5가지 요약 수치(최소값, 제 1사분위값, 제 2사분위값, 제 3사분위값, 최대값)를 가지고 그리프를 만듭니다.

```
In [ ]: #나이대별 총이용금액 분포를 박스 그래프로 그리기
plt.boxplot(df['age_itg_cd'])
plt.show()
```

• ##### 상관관계(Heatmap) : Seaborn 상관관계를 확인하기 위해서 주의해야 할 점은 모든 데이터가 'continous'해야 한다는 점입니다.

corr()은 상관관계를 측정 할 수 있는 함수입니다.

```
In [ ]: df.corr()
```

```
In [ ]: #컬럼별 상과관계를 heatmap 그래프로 그리기
plt.figure(figsize=(12, 12))
sns.heatmap(df.corr(), annot=True)
plt.show()
```

③ 차트 꾸미기

- 제목과 축 레이블 추가하기

Plot에 X,Y 축 레이블이나 제목을 붙이기 위해서는 plt.xlabel(축이름), plt.ylabel(축이름), plt.title(제목) 등의 함수를 사용합니다.

```
In [ ]: #주간 최고기온 그래프에 표제목과 X축, Y축 이름 넣기
plt.figure()
plt.plot(["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"], [28,30,29,31,32,31,31] )
plt.xlabel('Day')
plt.ylabel('Temp')
plt.title('High Temperature')
plt.show()
```

- 범례 추가하기

플롯에 여러 개의 라인들을 추가하기 위해서는 plt.plot()을 plt.show() 이전에 여러 번 사용하면 됩니다.

또한, 각 라인에 대한 범례를 추가하기 위해서는 plt.legend() 함수를 사용하여 각 라인에 대한 범례를 순서대로 지정합니다.

```
In [ ]: #두개선을 가지는 임의의 선그래프를 그리고 범례 추가하기
plt.plot([1,2,3], [1,4,9])
plt.plot([2,3,4],[5,6,7])
plt.xlabel('Quarter')
plt.ylabel('Score')
plt.title('Game Result')
plt.legend(['A team', 'B team'])
plt.show()
```

[실습2] 데이터 시각화 하기

[Q] 'df' 데이터에서 고객의 나이('age_itg_cd') 분포를 확인해 보세요.

[조건]

20개 구간으로 나누고, X축에는 '나이', Y축에는 '고객 수'로 축 이름을 설정합니다.

```
In [ ]: # 'age_itg_cd'에 대한 빈도를 20개 구간으로 구간으로 그리기
plt.figure()
plt.hist(df["age_itg_cd"], bins=20)
plt.xlabel('나이')
plt.ylabel('고객 수')
plt.show()
```

[Q] 'df' 데이터에서 'voc trt need time itg cd'(VOC 처리 소요시간 통합코드)와 'voc sttus itg cd'(VOC 상태 통합코드)의 관계를 확인하는 산점도 그래프를 그려봅시다.

[조건]

그래프의 크기는 (6,6)으로 설정합니다.

X축에는 'VOC처리 소요시간', Y축에는 'VOC 상태'로 축 이름을 설정합니다.

```
In [ ]: plt.figure(figsize=(6,6))
sns.scatterplot(x=df['voc_trt_need_time_itg_cd'], y=df['voc_sttus_itg_cd'])
plt.xlabel('VOC 처리소요시간')
plt.ylabel('VOC 상태')
plt.show()
```

4. Feature Engineering

- ① Binning
- ② Encoding

① Binning

Binning은 단어 뜻 그대로 자료를 일정한 규격의 통에 넣는것 입니다.

만약 관측치가 연속형이면서 범위가 너무 다양할 경우, 적절히 그룹을 지어주면 데이터를 이해하기가 더 쉬워질수 있기에 사용합니다.

즉, 연속형 변수를 범주형 변수로 만드는 방법이라고 보시면 됩니다.

```
In [ ]: df['age_itg_cd'].describe()
```

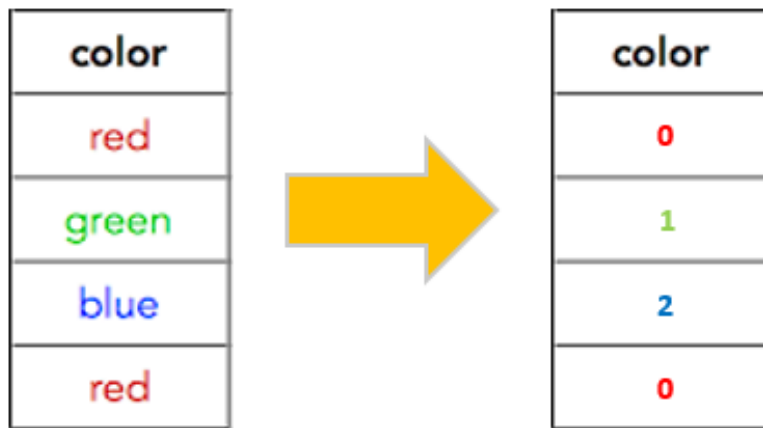
```
In [ ]: # 'age_itg_cd'를 활describe 나이대("by_age") Feature 만들기
df['by_age']=df['age_itg_cd']//10*10
df=df.astype({'age_itg_cd': int, 'by_age':int})
```

```
In [ ]: df.head()
```

② Encoding

기계 학습의 주요 문제 중 하나는 많은 알고리즘이 범주 형 데이터를 입력값으로 수용하지 않는다는 점인데, 이를 Encoding을 통해 해결 할 수 있습니다.

- **Label Encoding** : 문자를 숫자로 변환



문자형 카테고리를 숫자형 카테고리 값으로 변환합니다.

하지만 레이블 인코딩이 일괄적으로 숫자 값으로 변환이 되면서 몇몇 ML알고리즘에는 이를 적용할 경우 예측 성능이 떨어지는 경우가 발생할 수 있습니다.

__숫자로 되어 있어 잘못하면 가중치로 인식하여 값에 왜곡이 생기게 됩니다.__

이러한 특성 때문에 레이블 인코딩은 선형 회귀에는 적용하지 않습니다.

```
In [ ]: # 라이브러리 임포트
        from sklearn.preprocessing import LabelEncoder
```

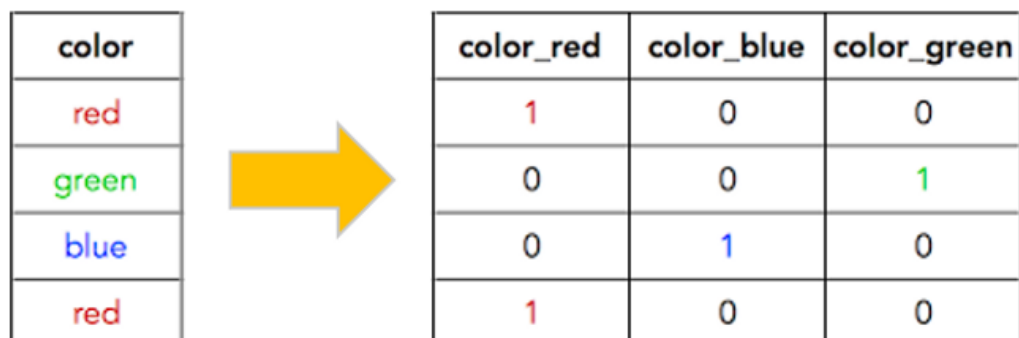
```
In [ ]: # LabelEncoder() 호출
        le = LabelEncoder()

        # LabelEncoding 결과를 새로운 컬럼을 생성하여 저장
        df['le_cust_clas_itg_cd'] = le.fit_transform(df['cust_clas_itg_cd'])
```

```
In [ ]: df.head()
```

```
In [ ]: # LabelEncoding 매핑값 확인
        le.classes_
```

- **One-Hot Encoding** : pandas 'get_dummies' 활용



원-핫 인코딩은 간단히 말해 한 개의 요소는 True, 나머지 요소는 False로 만들어 주는 기법입니다.

행 형태로 돼 있는 피처의 고유 값을 열 형태로 차원을 변환한 뒤,
고유 값에 해당하는 칼럼에만 1을 표시하고 나머지 칼럼에는 0을 표시합니다.
Pandas의 get_dummies()를 사용하면 간단하게 원-핫 인코딩을 진행할 수 있습니다.

```
In [ ]: #pandas에서는 get_dummies함수를 사용하면 쉽게 One-Hot Encordng이 가능
pd.get_dummies(df['cust_clas_itg_cd'])
```

```
In [ ]: df_fix.head()
```

```
In [ ]: df = df_fix.copy()
```

[실습3] 데이터 Feature Engineering 하기

[Q] 'df' 데이터에서 'age_itg_cd'를 5단위로 범주화하고 'by_age'컬럼에 저장하세요.

['by_age']: 나이는 0 ~ 4세는 0 / 5 ~ 9세는 5 / 10 ~ 14세는 10으로 5단위 범주화

```
In [ ]: df['by_age']=df['age_itg_cd']//5*5
df=df.astype({'age_itg_cd': int, 'by_age':int})
```

```
In [ ]: df.head()
```

[Q] 'df' 데이터에서 'trm_yn' 컬럼은 Label Encoding 하세요.

[조건]

새로운 컬럼을 생성하지 말고 기존의 컬럼에 덮어쓰기 하세요.

```
In [ ]: #라이브러리 임포트
from sklearn.preprocessing import LabelEncoder

# LabelEncoder() 호출
le = LabelEncoder()

df['trm_yn'] = le.fit_transform(df['trm_yn'])
```

```
In [ ]: df.head()
```

[Q] 'df' 데이터에서 나머지 범주형 데이터는 pandas get_dummies를 활용하여 One-Hot Encoding 후 저장하세요.

[조건]

파라미터 'drop_first=True', 'columns =['컬럼명', '컬럼명...]' 를 적용하세요.

```
In [ ]: # 범주형(object) 컬럼명 리스트
object_columns = ['voc_type_itg_cd',
                  'voc_sttus_itg_cd',
                  'cust_clas_itg_cd',
                  'cont_sttus_itg_cd',
                  'cust_dtl_ctg_itg_cd',
```

```
'voc_trt_degr_div_itg_cd',  
'voc_trt_need_time_itg_cd']
```

```
In [ ]: df = pd.get_dummies(df, columns=object_columns, drop_first=True)
```

```
In [ ]: df.head()
```

```
In [ ]: # 다음 실습을 위한 데이터 저장  
df.to_csv(DATA_PATH + '/voc_rcp_practice3.csv', index=False)
```